

I D E N T I T Y L A B S E R I E S

The Full SCIM Lifecycle

Provisioning, Updating, and Deprovisioning Users
with Okta, SCIM 2.0, and a Structured Audit Trail

Jamie Ruth

July 2026

Demonstration Build

[Part 2 of 3](#)

Executive Summary

This document describes a complete, working SCIM 2.0 identity lifecycle built from scratch: an Okta directory connected to a custom SCIM server through a secure tunnel, with every operation recorded in a structured audit trail. The build proves the three operations that define enterprise user lifecycle management: provisioning (a user joins), updating (a user changes roles), and deprovisioning (a user loses access).

The system is not a simulation. It runs against a live Okta Integrator org, handles real HTTP calls, enforces bearer-token authentication, and writes a machine-readable audit log for every operation. The audit trail captures who performed the action (via token fingerprint), what changed, and how the server responded.

The full lifecycle was proven across two sessions. Provisioning and deprovisioning were validated on July 9, 2026. The update operation was validated on July 10, completing the **joiner-mover-leaver** cycle that every enterprise identity team manages at scale.

Environment Architecture

The diagram below shows the five layers of the lab environment and how they connect, from the company directory down to the audit trail.

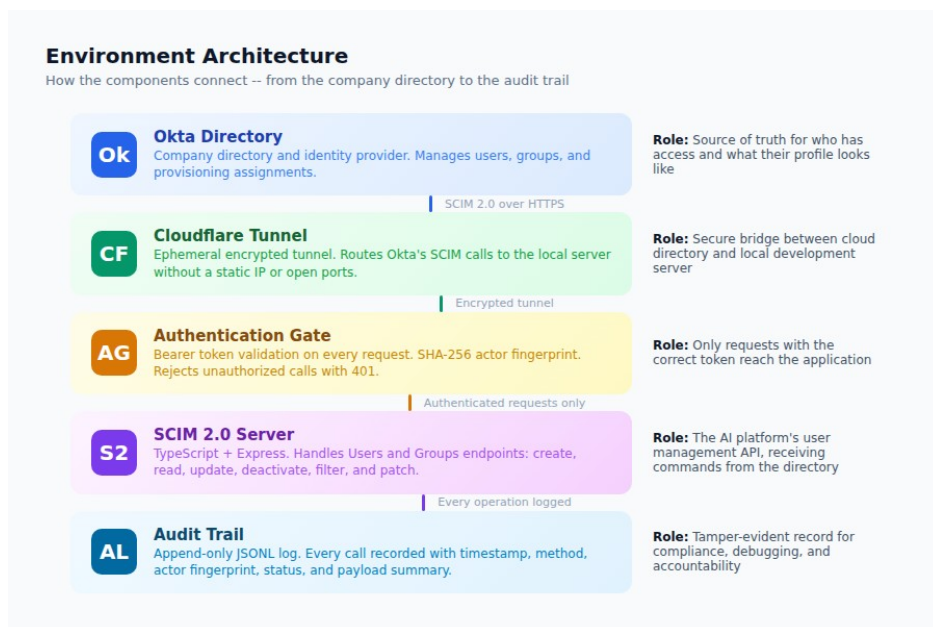


Figure 1: Environment architecture -- Okta directory to audit trail

Build Timeline

The lab was built and tested in six phases across two days. Each phase added a layer to the architecture and was validated before proceeding to the next.

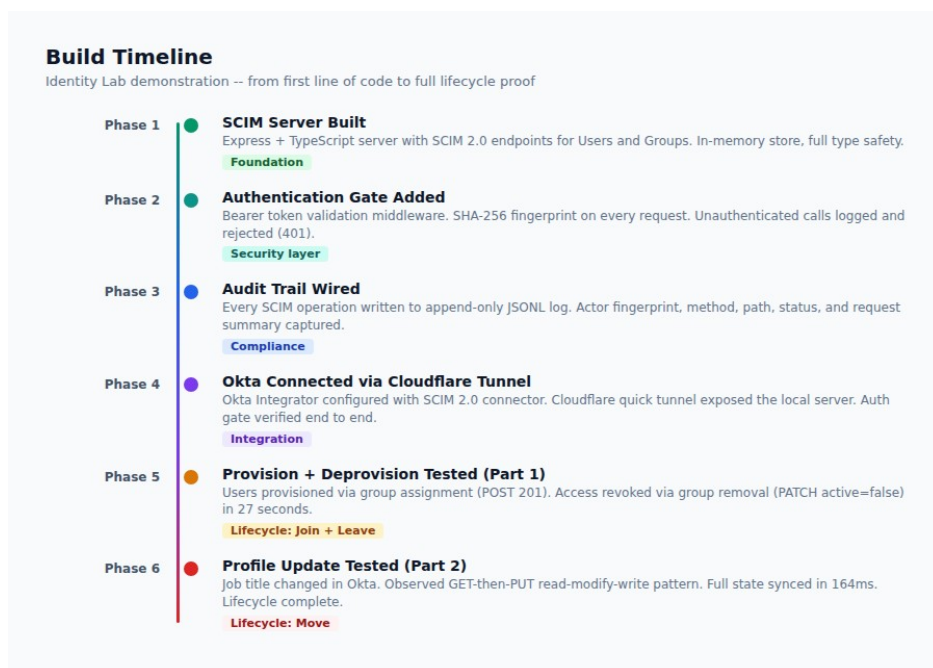


Figure 2: Build timeline -- six phases from foundation to full lifecycle proof

1. What Was Built

The build consists of five components working together:

SCIM 2.0 Server

A Node.js application implementing the SCIM 2.0 protocol (RFC 7644) for user resource management. The server handles four HTTP methods on the `/scim/v2/Users` endpoint: POST for creating users, GET for reading user records, PUT for full-record updates, and PATCH for partial modifications (used by Okta for deprovisioning). Users are stored in a local JSON file with SCIM-compliant schema attributes including `userName`, `displayName`, `name`, `emails`, `title`, and `active` status.

Bearer Token Authentication

Every inbound request must include a valid Authorization header with the Bearer scheme. Requests without a token or with an invalid token receive a 401 Unauthorized response. The server computes a SHA-256 fingerprint of each valid token and logs it as the actor identity in the audit trail. This means the audit log proves which integration made each call without storing the token itself.

Structured Audit Trail

Every SCIM operation writes a single JSON line to `logs/audit.jsonl`. Each entry includes: the ISO 8601 timestamp, the HTTP method, the request path, the actor fingerprint (first 12 hex characters of the token hash), the relevant request body fields, and the HTTP status code returned. The log is append-only, machine-readable, and designed for compliance review.

Okta Integration

The SCIM server is registered as a private SCIM integration ("AI Platform (Demonstration)") inside an Okta Integrator Free Plan org. Okta is configured to push user provisioning events to the server whenever users are assigned to or removed from the `ai-power-users` group. The integration uses Okta's Header Authentication mode with a bearer token.

Cloudflare Tunnel

Since the SCIM server runs locally, a Cloudflare quick tunnel (`cloudflared`) exposes it to Okta over HTTPS. The tunnel requires no inbound firewall rules, no static IP, and no DNS configuration. Okta sends its SCIM calls to the tunnel URL, which forwards them to `localhost:3010`.

2. The Three Operations

The SCIM lifecycle consists of three operations that map directly to the joiner-mover-leaver model used in enterprise identity management.

2.1 Provision (Joiner)

What happens: A user is assigned to the ai-power-users group in the Okta admin console. Okta detects the assignment and sends a POST request to the SCIM server's /scim/v2/Users endpoint with the user's profile attributes. The server creates the user record and returns 201 Created.

What was tested: Two test users (Ada Test and Grace Test) were assigned to ai-power-users. Okta pushed both as POST requests. The server returned 201 for each, and the audit trail recorded both with the same actor fingerprint.

Audit evidence:

```
POST /scim/v2/Users -> 201 Created
Actor: cda486dca4a9 | Timestamp: 2026-07-09T21:08 UTC
```

2.2 Update (Mover)

What happens: A profile attribute is changed in the Okta admin console (for example, a job title). Okta does not send a PATCH with just the changed field. Instead, it performs a read-modify-write: it sends a GET to retrieve the current user record from the SCIM server, merges the updated attribute into the full profile, and sends a PUT with the complete user object. This means the SCIM server always receives the entire user state, not a fragment.

What was tested: Grace Test's Title field was changed to "AI Platform Analyst" in the Okta admin console. Within milliseconds, the audit trail showed a GET followed by a PUT on the same user ID. The PUT payload included all profile fields with the updated title. The server returned 200 for both requests.

Audit evidence:

```
GET /scim/v2/Users/8e9b5346-... -> 200 | 2026-07-10T08:49:26.213Z
PUT /scim/v2/Users/8e9b5346-... -> 200 | 2026-07-10T08:49:26.377Z
Actor: cda486dca4a9 (same fingerprint as provision and deprovision)
```

Technical observation: The 164ms gap between the GET and PUT confirms this is a synchronous read-modify-write, not an asynchronous push. This pattern has a practical benefit: you can diff any two consecutive PUTs for the same user and see exactly what changed, because each PUT contains the complete user state at that moment.

2.3 Deprovision (Leaver)

What happens: A user is removed from the ai-power-users group (or unassigned from the SCIM app) in Okta. Okta sends a PATCH request that sets the user's active attribute to false. The server marks the user as inactive but preserves the record for compliance purposes.

What was tested: Ada Test was removed from ai-power-users. Approximately 27 seconds later, the audit trail showed a PATCH with op: replace on Ada's user ID. A subsequent authenticated GET confirmed the user record still existed but with active: false.

Audit evidence:

```
PATCH /scim/v2/Users/f87924e5-... -> 200 | 2026-07-09T21:36:12Z
Operations: [{op: replace}] (active set to false)
Actor: cda486dca4a9 (same fingerprint throughout)
```

3. The Actor Fingerprint

One detail ties the entire lifecycle together: the actor fingerprint `cda486dca4a9` appears on every operation across both sessions. This single value proves that all three operations (provision, update, deprovision) were performed by the same Okta integration, using the same bearer token, against the same SCIM server.

The fingerprint is derived from the first 12 hex characters of a SHA-256 hash of the bearer token. It serves the same purpose as an API key ID: it identifies the caller without exposing the credential. In a production environment, this fingerprint would allow a security team to trace any SCIM operation back to the specific integration that performed it.

4. Authentication Gate

The SCIM server enforces bearer-token authentication on every endpoint. This was validated before and after each lifecycle test:

Test	Method	Expected	Actual
Authenticated request	GET with valid Bearer token	200 OK	200 OK
Unauthenticated request	GET with no Authorization header	401 Unauthorized	401 Unauthorized
Invalid token	GET with wrong Bearer token	401 Unauthorized	401 Unauthorized

Okta quirk discovered: Okta's SCIM 2.0 Test App (Header Authentication mode) sends the Authorization header as the raw field value. It does not prepend "Bearer ". If you enter just the token string, the server receives an Authorization header without the Bearer scheme and returns 401. The fix is to enter "Bearer <token>" (with the prefix) in Okta's API Token field. This was diagnosed using the audit trail: requests from Okta showed actor: "none" (no valid Bearer prefix found), while curl requests with the correct header showed the expected fingerprint.

5. Key Findings

Okta uses GET-then-PUT for updates, not PATCH. The SCIM RFC (7644) defines both PUT and PATCH. Okta uses PATCH only for deprovisioning (setting active to false). For profile attribute changes, it performs a read-modify-write: GET the full user, merge the change, PUT the full user back. This means the server always receives the complete state on every update. It also means the audit trail contains full snapshots, which makes debugging and compliance review significantly easier.

The audit trail diagnosed a problem faster than Okta's own UI. When the initial integration test failed (Okta's "Test API Credentials" returned unauthorized), the Okta admin console provided no actionable diagnostic. The audit log showed the root cause in one line: requests from Okta had actor: "none" (no valid Bearer prefix), while manual curl requests showed the expected fingerprint. The fix (prepending "Bearer " in the token field) took thirty seconds once the log pointed to the cause.

Deprovisioning latency is measurable. The gap between removing a user from a group in the Okta UI and the PATCH arriving at the SCIM server was approximately 27 seconds. This is useful context for operations teams: it sets expectations for how quickly access revocation propagates through the system.

Update latency is sub-second. The GET-then-PUT cycle for a profile update completed in 164 milliseconds. The directory change and the server update are effectively synchronous from a user-experience perspective.

6. What Comes Next

This build proves the SCIM lifecycle. The next steps extend it:

Single sign-on (SSO): Wiring OIDC authentication from the same Okta org into a live application, so the identity provider controls both provisioning and authentication.

MCP server for identity operations: An AI assistant that can provision and deprovision users through the SCIM server, with human approval required on any destructive action and an audit line on every call. This is the intersection of identity management and AI platform tooling.

Cross-directory testing: Repeating the same lifecycle on Microsoft Entra ID, proving the architecture is directory-agnostic.

This document describes a demonstration build. All test users, groups, and credentials are lab-only resources.